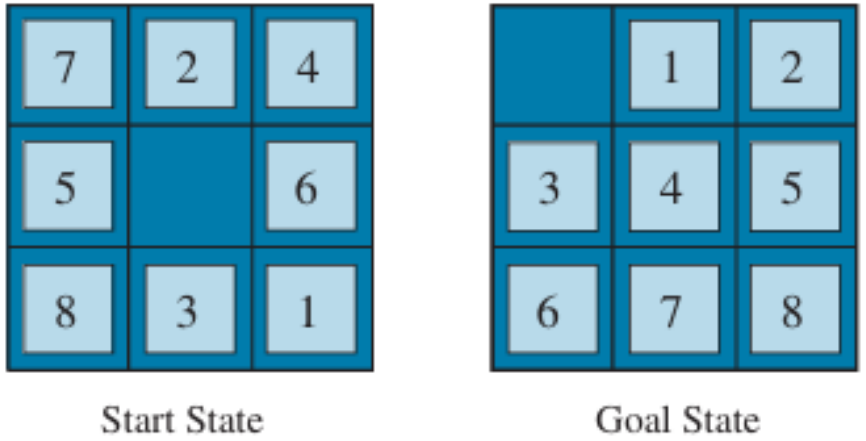
Bài 3. Solving Problems by Searching

Mục tiêu

- Kiến thức nền tảng về **tìm kiếm trong AI**.
- Hiểu được **sự đánh đổi (trade-offs)** giữa thời gian, bộ nhớ và chất lượng nghiệm trong các thuật toán tìm kiếm.
- Nắm được **hai nhóm thuật toán chính**:
  - **Uninformed search** (tìm kiếm không có thông tin).
  - **Informed/Heuristic search** (tìm kiếm có thông tin).
- Phát triển **tư duy thuật toán** qua việc cài đặt, trực quan hóa, và so sánh các thuật toán.
- Ứng dụng các kỹ thuật tìm kiếm vào **bài toán thực tế** (tìm đường trong maze).

1. Vai trò của tìm kiếm

- **Tìm kiếm** là phương pháp cơ bản để chọn chuỗi hành động từ trạng thái ban đầu → trạng thái mục tiêu.
- Được dùng khi **không có công thức trực tiếp** để tìm lời giải.
- Ứng dụng rộng rãi:
- Tìm đường đi ngắn nhất (Google Maps, robot navigation).
- Giải trò chơi (chess, sudoku, 8-puzzle).
- Lập kế hoạch (planning).



A typical instance of the 8-puzzle.

2. Problem-Solving Agent Framework

Một problem gồm 5 yếu tố chính:

1. **Initial state (Trạng thái ban đầu)** – ví dụ: vị trí robot tại ô (0,0).
2. **Actions (Hành động)** – ví dụ: lên, xuống, trái, phải.
3. **Transition model (Mô hình chuyển trạng thái)** – định nghĩa kết quả của mỗi hành động.
4. **Goal test (Bài kiểm tra mục tiêu)** – ví dụ: "đến được ô (N,M)".
5. **Path cost (Chi phí hành động)** – có thể là số bước, độ dài đường đi, hoặc tổng chi phí.

3. State Space Graph và Solution

Khái niệm:

•**State space**: tập tất cả trạng thái có thể đạt được từ initial state bằng cách áp dụng các hành động hợp lệ.

•**State space graph**: đồ thị có:

- Node = trạng thái
- Edge = hành động

•**Solution**: một **path** từ initial state → goal state.

Ví dụ minh họa:

•8-puzzle: mỗi node là trạng thái bàn cờ, cạnh là hành động trượt ô số.

•Maze: mỗi node là một ô vuông, cạnh là di chuyển.

4. Tiêu chí đánh giá giải thuật

- **Completeness (Hoàn chỉnh)**: có đảm bảo tìm thấy nghiệm nếu tồn tại?
- **Optimality (Tối ưu)**: nghiệm tìm được có chi phí nhỏ nhất?
- **Time complexity**: mất bao lâu (tính theo số node sinh ra)?
- **Space complexity**: tốn bao nhiêu bộ nhớ để lưu trữ node trung gian?
- **Ví dụ đối chiếu:**

  BFS: Hoàn chỉnh, tối ưu với chi phí đơn vị, nhưng tốn bộ nhớ.

  DFS: Ít tốn bộ nhớ, nhưng không đảm bảo tìm được nghiệm tối ưu.

5. Các Thuật toán Uninformed Search

- Breadth-First Search (BFS)
- Uniform-Cost Search (UCS)
- Depth-First Search (DFS)
- Depth-Limited & Iterative Deepening (IDS)
- Bidirectional Search

5.1 Breadth-First Search (BFS)

- Dùng **priority queue** theo **g(n)**.
- Khác BFS ở chỗ: mở rộng theo chi phí đường đi chứ không phải độ sâu.
- Luôn tìm nghiệm có chi phí nhỏ nhất.
- **Ví dụ Romania**: Sibiu → Rimnicu Vilcea → Pitesti → Bucharest tốt hơn Sibiu → Fagaras → Bucharest.
- **Độ phức tạp**: $O(b^{1+\lfloor C/\epsilon \rfloor}$

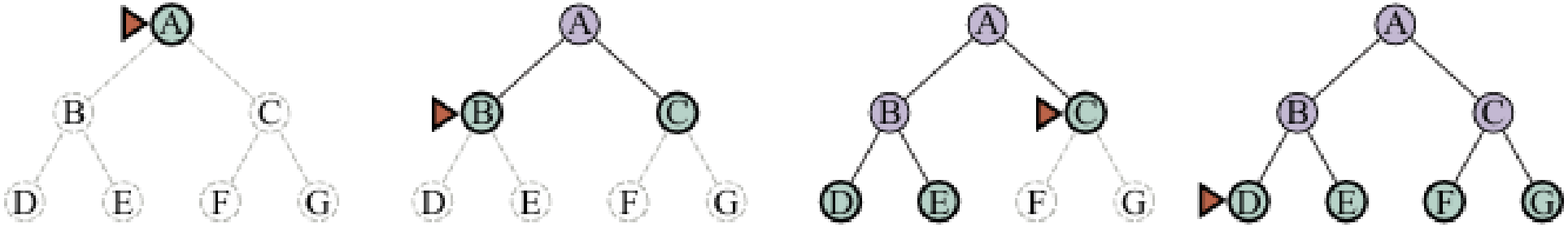
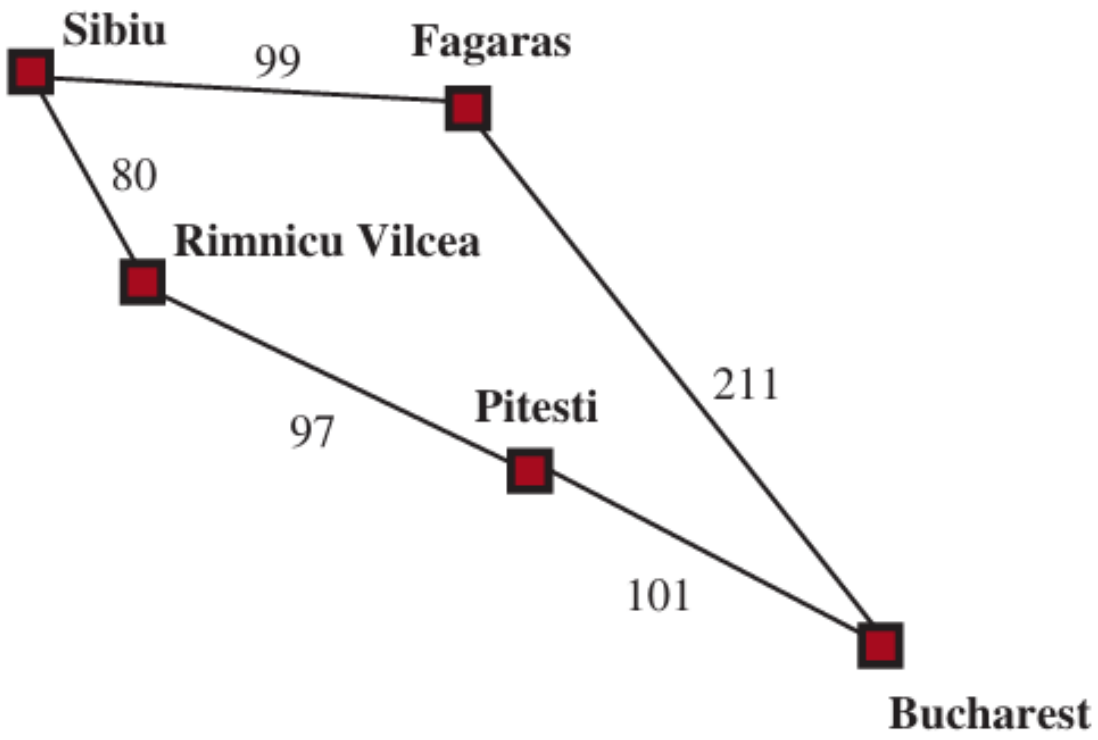


Figure 3.8 Breadth-first search on a simple binary tree. At each stage, the node to be expanded next is indicated by the triangular marker.

5.2 Uniform-Cost Search (UCS)- Dijkstras algorithm

- Dùng **priority queue** theo **g(n)**.
- Khác BFS ở chỗ: mở rộng theo chi phí đường đi chứ không phải độ sâu.
- Luôn tìm nghiệm có chi phí nhỏ nhất.
- **Ví dụ Romania**: Sibiu → Rimnicu Vilcea → Pitesti → Bucharest tốt hơn Sibiu → Fagaras → Bucharest.
- **Độ phức tạp**: $O(b^{1+\lfloor C/\epsilon \rfloor})$



5.3 Depth-First Search (DFS)

Ý tưởng: mở rộng **nút sâu nhất** trước.
Dùng stack (LIFO).
Tính chất:

- Ít tốn bộ nhớ (O(bm), m = độ sâu tối đa).
- Không hoàn chỉnh nếu đồ thị có vòng.
- Không tối ưu (có thể tìm được nghiệm xa hơn).

Ưu điểm:
Dễ cài đặt, bộ nhớ nhỏ.
Nhược điểm:
Có thể rơi vào vòng lặp vô hạn.
Có thể tìm nghiệm kém tối ưu.

5.3 Depth-First Search (DFS)

1. **Bắt đầu từ A** → mở rộng sang trái: B, rồi D.
2. DFS tiếp tục đi sâu: từ D xuống H.
3. H không phải goal → quay ngược lại (backtrack).
4. Mở rộng tiếp I (anh em của H).
5. I không phải goal → quay lại D → D hết nhánh.
6. Quay về B → duyệt sang E → J, K.
7. Tương tự, DFS thử hết nhánh trái trước (J, rồi K).
8. Khi không thấy kết quả, backtrack ngược lên để thử tiếp.
9. Cuối cùng DFS sang bên phải (C, F) rồi tiến dần đến M.

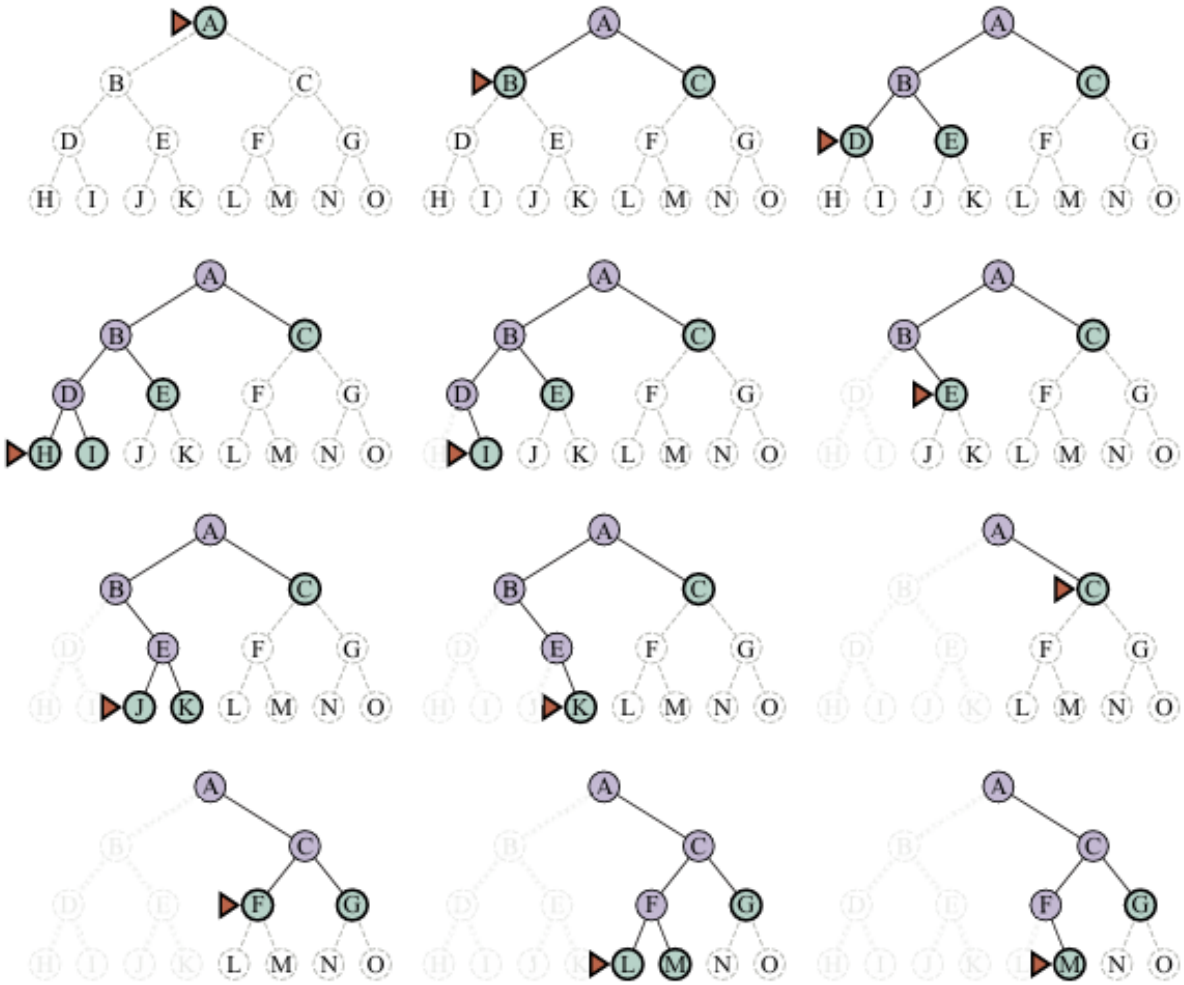
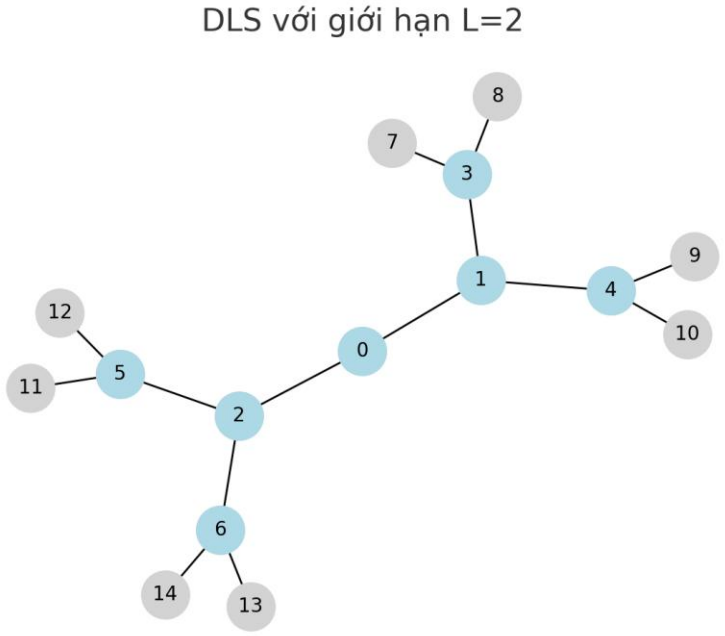
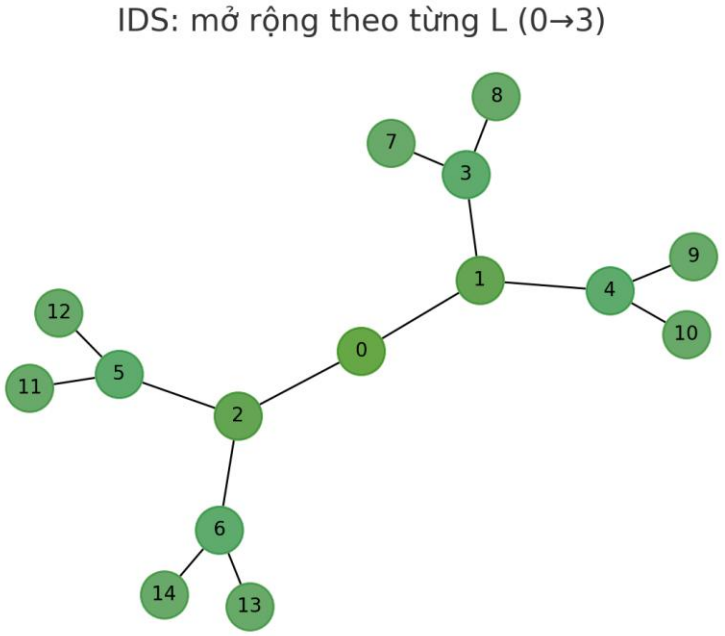


Figure 3.11 A dozen steps (left to right, top to bottom) in the progress of a depth-first search on a binary tree from start state A to goal M. The frontier is in green, with a triangle marking the node to be expanded next. Previously expanded nodes are lavender, and potential future nodes have faint dashed lines. Expanded nodes with no descendants in the frontier (very faint lines) can be discarded.

5.4 Depth-Limited Search (DLS) và Iterative Deepening (IDS)

- **DLS**: đặt giới hạn độ sâu L để tránh đi mãi.
- Không hoàn chỉnh nếu L < độ sâu nghiệm.
- **IDS**: chạy DFS nhiều lần với L tăng dần.
- Bộ nhớ thấp O(bd),tối ưu như BFS.
- Tái tạo node ở trên nhiều lần nhưng chi phí không đáng kể vì phần lớn node nằm ở mức cuối.





5.3 Bidirectional Search

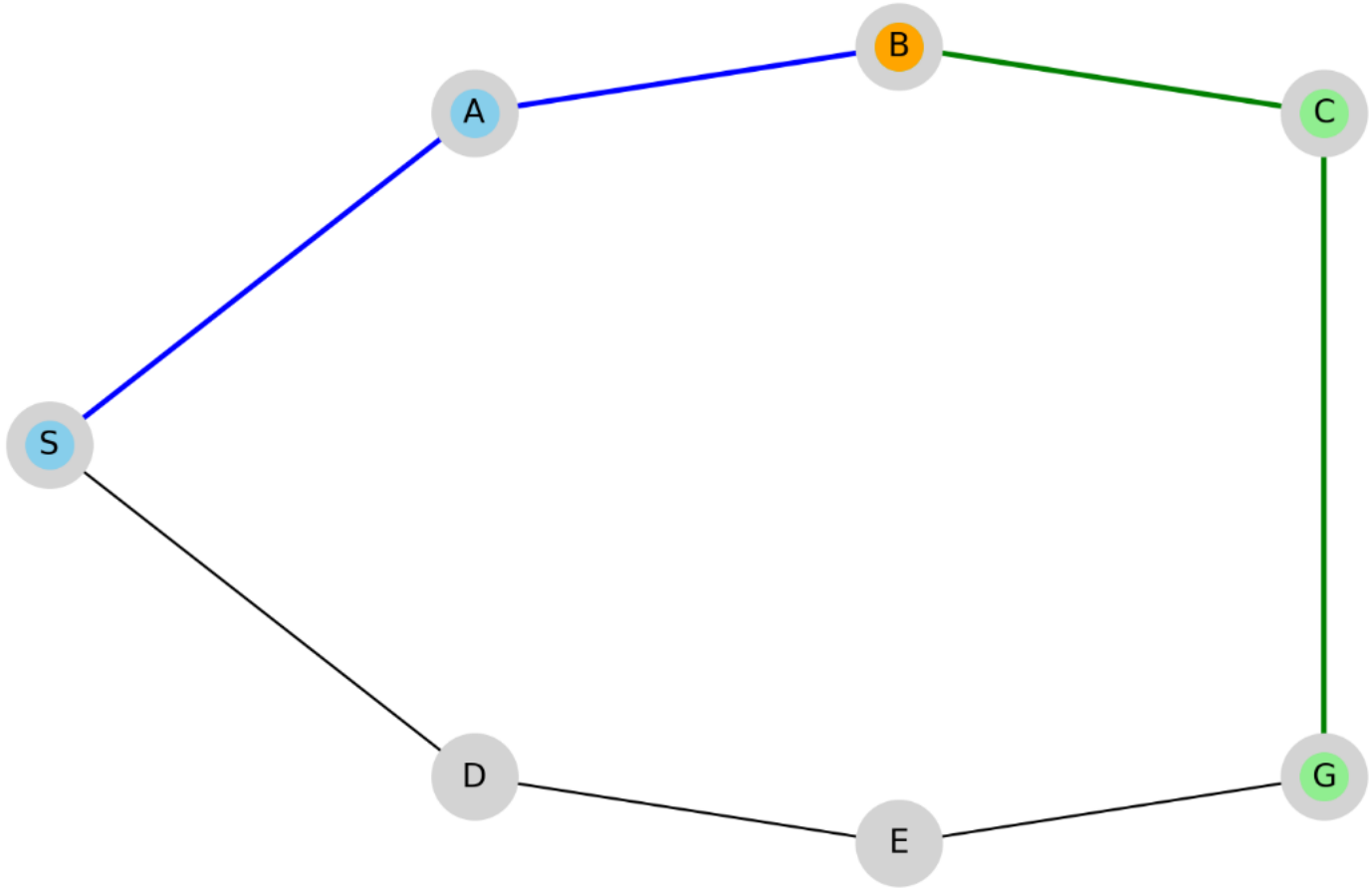
Ý tưởng: tìm kiếm từ **start** và **goal** cùng lúc.
Dừng khi hai biên gặp nhau.
Hiệu quả nếu có thể xác định trạng thái goal cụ thể.

Tính chất:

- Hoàn chỉnh.
- Tối ưu với chi phí đơn vị.
- Tiết kiệm thời gian so với BFS (O(b^(d/2)) thay vì O(b^d)).

Hạn chế:

Khó áp dụng nếu goal test không xác định rõ ràng.
Cần bộ nhớ lớn để lưu cả hai frontier.



6.Informed Search và Heuristics

6.1 Khái niệm **Heuristic**

Khái niệm Heuristic

Định nghĩa:

Heuristic là **hàm ước lượng** (estimate function), ký hiệu h(n), cho biết **chi phí ước lượng** từ trạng thái hiện tại **n** đến trạng thái đích **(goal).**

Đây **không phải chi phí thật sự**, mà chỉ là một cách "dự đoán thông minh" để hướng dẫn tìm kiếm.

Ký hiệu: h(n)

Ví dụ:

Trong lưới 2D (maze), heuristic thường dùng là **Manhattan distance**:

$$h(n) = |x_{goal} - x_n| + |y_{goal} - y_n|$$

(Tổng số bước đi theo hàng và cột).

Đây là ước lượng hợp lý nếu chỉ được phép đi 4 hướng (trên, dưới, trái, phải).

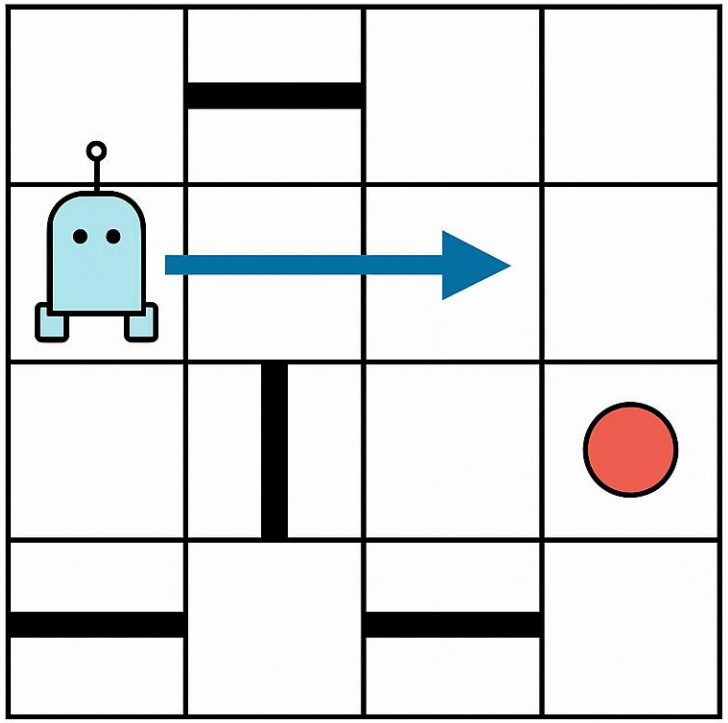
6.2 Greedy Best-First Search

Ý tưởng: Luôn chọn node “trông gần goal nhất” theo heuristic.
Công thức: f(n)=h(n)
Ưu điểm:
- Nhanh, tìm ra nghiệm sớm.
- Dùng ít bộ nhớ hơn so với tìm kiếm mù.

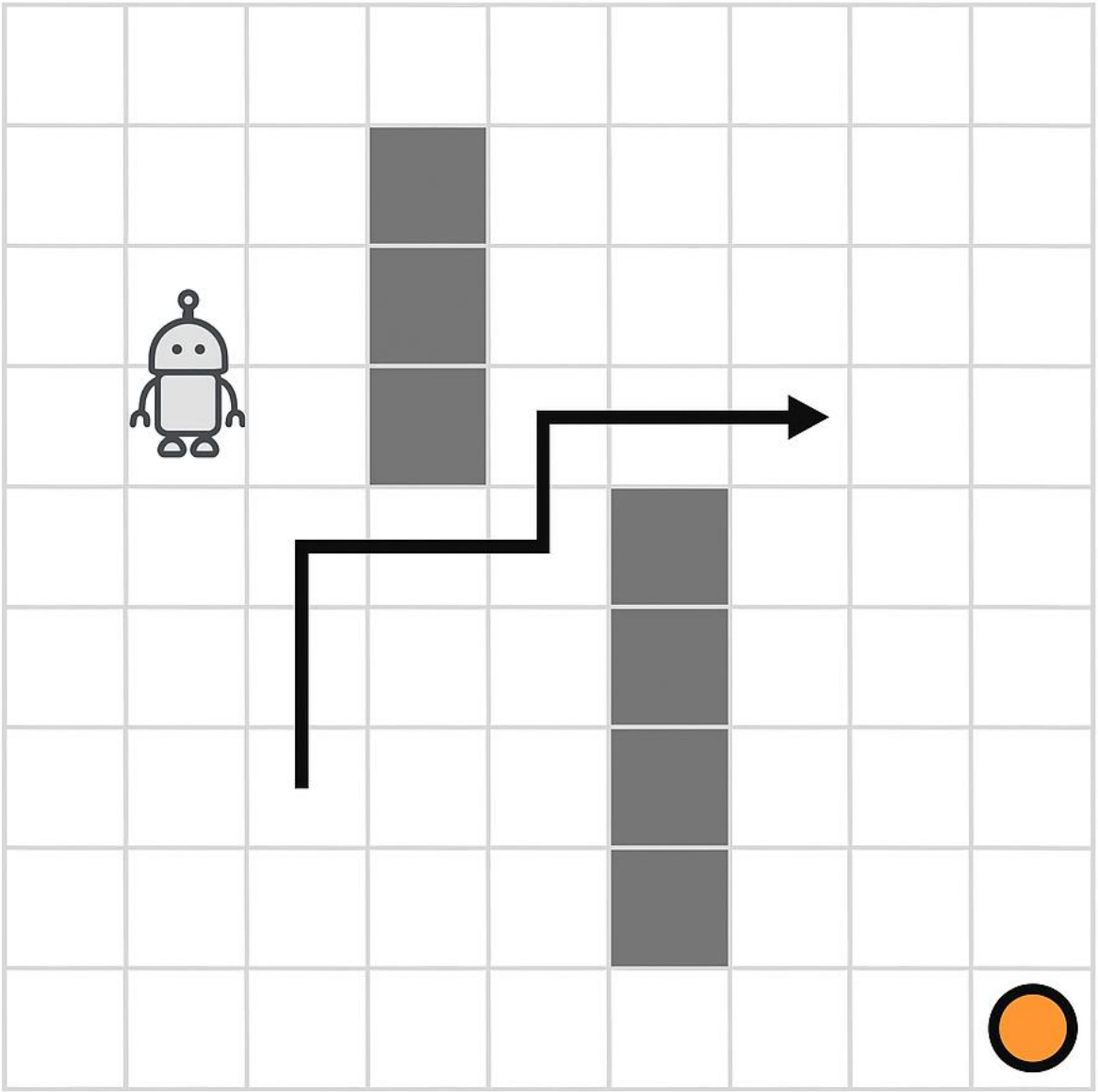
Nhược điểm:
- Không tối ưu (đường đi tìm được có thể không ngắn nhất).
- Có thể **lạc hướng** nếu heuristic đánh giá sai.

Minh họa:
- Robot trong mê cung: luôn chọn bước giảm khoảng cách Mar nhiều nhất.
- Nếu gặp tường, robot có thể bị kẹt hoặc đi vòng xa.

Robot in maze: always picks step that reduces Manhattan distance most



Greedy Best-First Search



6.3 A* Search

Ý tưởng: Kết hợp giữa chi phí đã đi và chi phí ước lượng còn lại.

Công thức:

f(n)=g(n)+h(n)

Trong đó:

g(n): chi phí **thực tế** từ start đến node n.

h(n): chi phí **ước lượng** từ n đến goal.

Đặc điểm:

Nếu heuristic **admissible (**heuristic có thể chấp nhận được**)**, A* sẽ tìm đường đi tối ưu.

Ví dụ minh họa: Hình 3.18

6.4 Admissible & Consistent Heuristic

Admissible:
Định nghĩa:

$h(n) \le h^*(n)$

với $h^*(n)$ là chi phí thật sự tối thiểu từ n đến goal.

•Nghĩa là heuristic **không bao giờ đánh giá quá cao** chi phí còn lại.

•Ví dụ: khoảng cách Manhattan trong lưới 4 hướng là admissible (không vượt quá đường đi thật).

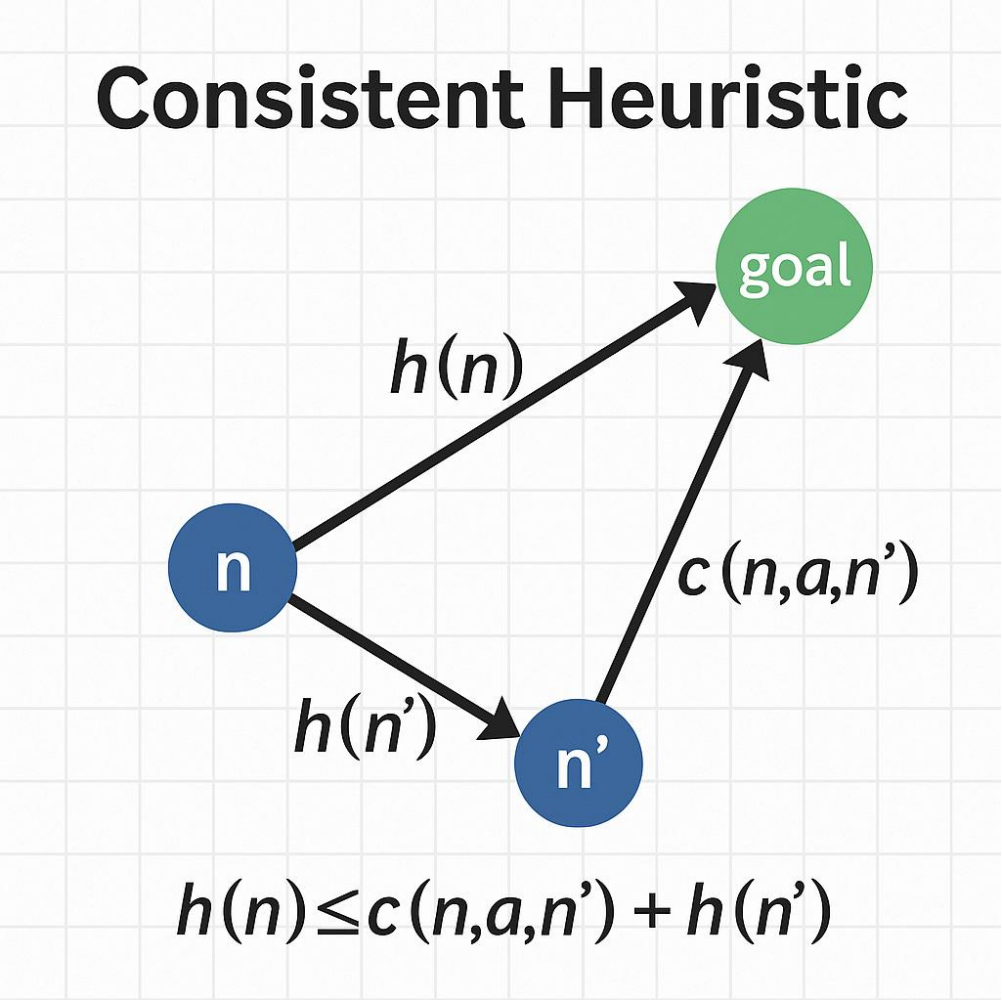
Consistent (hay Monotonic):
Định nghĩa:

$h(n) \le c(n,a,n') + h(n')$

với $c(n,a,n')$ là chi phí đi từ n đến láng giềng n′.

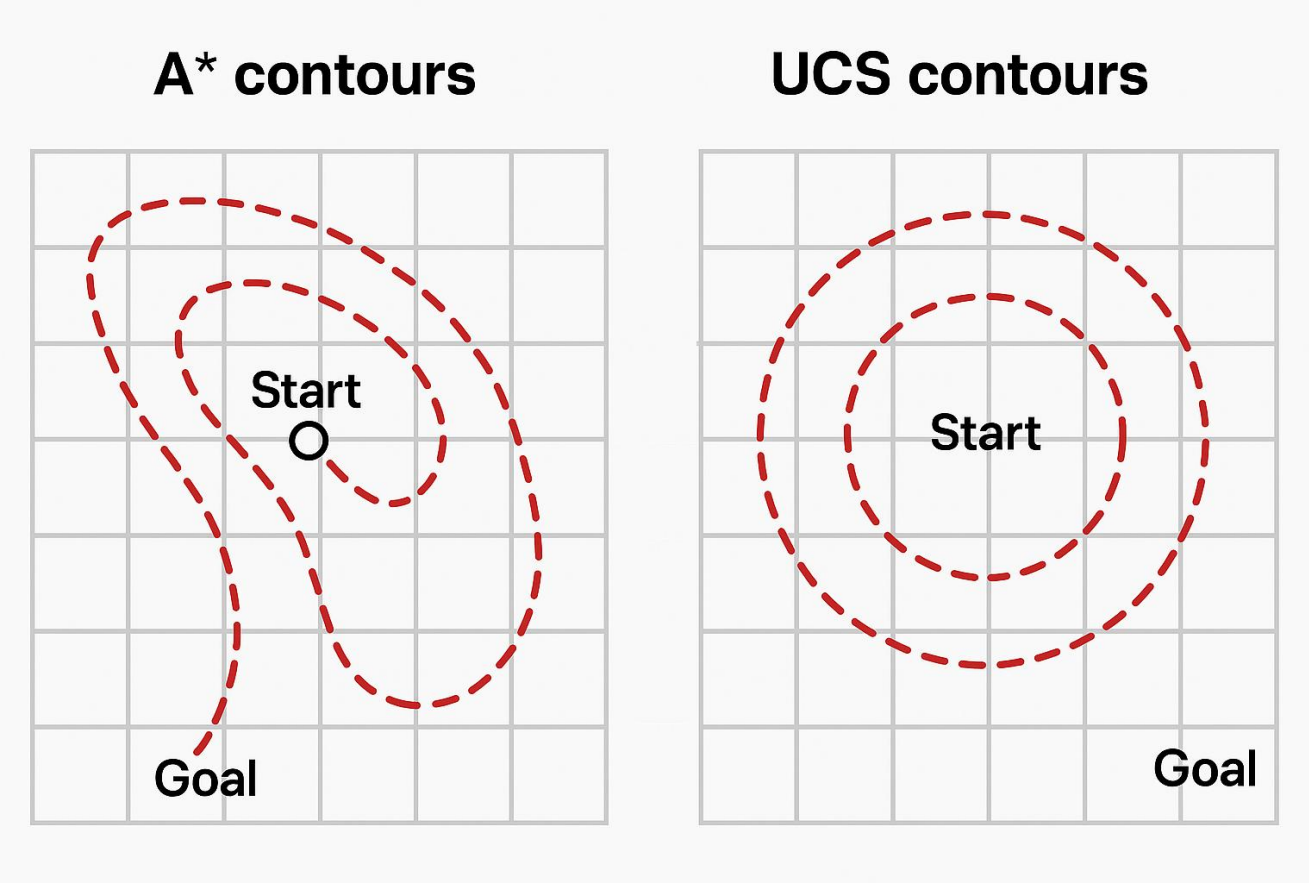
Ý nghĩa: khi đi từ n → n′, heuristic không "nhảy" bất hợp lý.

Đảm bảo A* không bị "lừa" và tìm nghiệm tối ưu.



6.5 Search Contours

- A* tạo **vùng đồng mức f(n)=g+h** (giống đường đồng mức địa hình)
- A*: contour kéo về phía goal → tập trung hơn, ít mở rộng hơn.



6.6 Weighted A* (Satisficing Search)

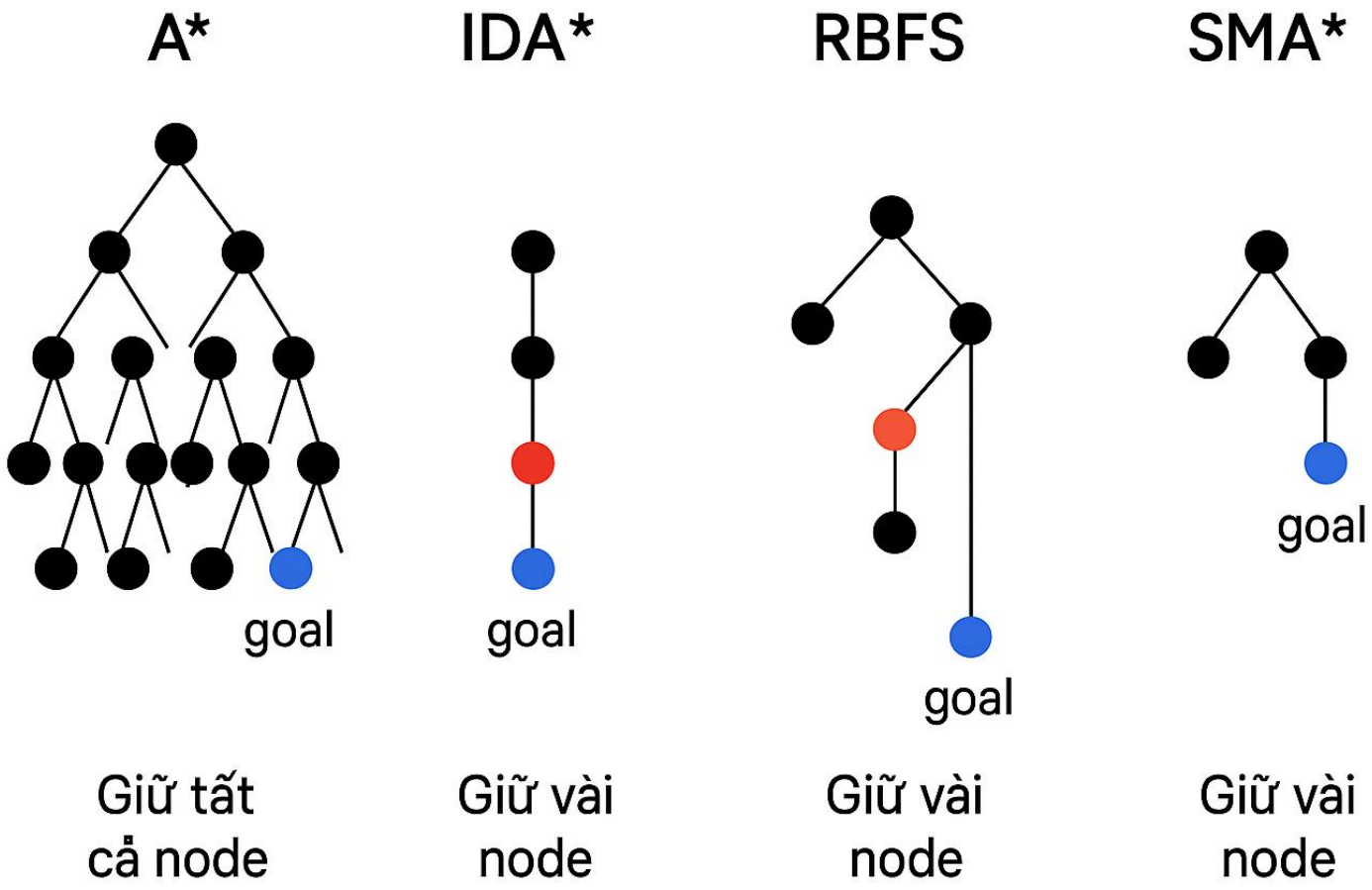
- Khi chấp nhận nghiệm “gần tối ưu” để đổi lấy tốc độ:

$f(n)=g(n)+W\cdot h(n), W>1$

- Kết quả: ít mở rộng node, nhanh hơn, nhưng nghiệm có thể dài hơn.
- Dùng trong các hệ thống thời gian thực (ví dụ GPS).

6.7 Memory-Bounded Search

- Vấn đề: A* tốn nhiều bộ nhớ.
- Các giải pháp:
  - **IDA***: iterative deepening theo f-cost.
  - **RBFS**: Recursive Best-First Search (linear memory).
  - **SMA***: Simplified Memory-Bounded A*.
- Tất cả giữ tính tối ưu nhưng tiết kiệm bộ nhớ.



6.8 Bidirectional Heuristic Search

- Ý tưởng: tìm kiếm từ cả **start** và **goal**, gặp nhau ở giữa.
- Lợi ích: giảm số node cần mở rộng (khi heuristic trung bình).
- Nhược điểm: khó chứng minh tối ưu, quản lý frontier phức tạp.
- Thực tế: hiệu quả nếu heuristic không quá mạnh.

6.9 Heuristic Functions

- Heuristic tốt quyết định hiệu quả.
- Ví dụ trong **8-puzzle**:

  h1: số quân sai vị trí (misplaced tiles).

  h2: tổng khoảng cách Manhattan của các quân.

  h2 **dominate** h1→ luôn tốt hơn.

6.10 Xây dựng Heuristic

- **Relaxed problems**: nới lỏng luật → lời giải tối ưu của bài toán đơn giản = heuristic admissible.
- **Pattern databases**: lưu chi phí tối ưu của subproblem, tra nhanh.
- **Landmarks**: chọn điểm mốc, tính chi phí qua mốc.
- **Learning heuristics**: huấn luyện từ dữ liệu bài toán đã giải.

Bài tập

Cài đặt và so sánh các thuật toán tìm kiếm